

AtCoder Beginner Contest Discord Reminder Bot 詳細設計書

1. 文書情報

項目	内容
文書名	AtCoder Beginner Contest Discord Reminder Bot 詳細設計書
対応する要件定義書	docs/atcoder_abc_discord_reminder_requirements.md
改訂日	2026-09-13
改訂内容	ESP32/TypeScript から Ubuntu Server/C++へ変更、通知本文を画像仕様へ変更
実行機体	Ubuntu Server を搭載した Dynabook パソコン(SSH ホスト: ryutoserver)
配置先	/srv/shared/remider(Windows 側の Y:¥remider に対応)
ソース管理場所	本リポジトリの開発ディレクトリ
基本言語	C++17
通知方式	Discord REST API(Gateway 不使用)
外部サーバー	使用しない
月額固定費	増加させない

2. 目的と設計方針

AtCoder Beginner Contest(ABC)の次回開催日時を AtCoder 公式ページから自動取得し、開催 2 時間前に Discord へ通知する。通知先は固定の Discord チャンネルとし、Ubuntu Server を搭載した Dynabook を常時稼働させて実行する。

今回の変更により、要件定義書に残る ESP32 固有の制約は次のように置き換える。

旧方針	新方針
ESP32 単体	Ubuntu Server を搭載した Dynabook 上の常駐サービス
TypeScript + Moddable SDK	C++17 + CMake
Arduino Preferences / NVS	SQLite3 によるローカル永続化
ESP32 向け軽量化・DOM なし解析	PC のメモリを利用した libxml2 HTML 解析
Wi-Fi 接続状態の確認	対象外。OS が提供するネットワークを利用し、接続確認処理は持たない
ESP32 再起動復元	systemd 再起動・OS 再起動後の SQLite 復元

2.1 優先順位

1. 誤ったコンテスト日時を通知しない
2. 通知失敗後に再送できる
3. プロセス再起動後も通知状態を復元できる
4. 同一通知の重複を抑止する
5. AtCoder へのアクセスを必要最小限にする
6. 実行速度を最適化する

実行速度より確実な動作を優先するため、HTML 解析と状態保存には実績のある外部ライブラリ・SQLite を使用する。HTML を推測で解析できたことにしたり、保存に失敗した状態を通知済みにしたりしない。

2.2 初期リリースの範囲

含める機能は次のとおりとする。

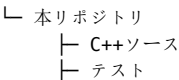
- ・ ABC のみの自動検出
- ・ 次回 ABC の選択
- ・ 開始 2 時間前の Discord 通知
- ・ 開催日時変更の検知
- ・ 通知後に日時が変更された場合の訂正通知
- ・ 通知済み状態の SQLite 保存
- ・ systemd による自動起動・異常終了時の再起動
- ・ Discord Gateway を使わない REST 通知

初期リリースには ARC、AHC、Web 設定画面、Discord からの設定変更、OTA、クラウドサーバー、外部 DB、外部監視サービスを含めない。

2.3 ソースとデプロイ先の分離

ソースコードは本リポジトリの開発ディレクトリで管理し、実行時に必要な成果物だけを SSH 経由でサーバーへデプロイする。開発用ソース、テスト、ビルド中間ファイルを実行環境へ直接配置しない。

開発環境



```

└─ ビルド成果物
    │
    │ SSH/SCPまたはrsync
    ▼
SSH接続先 ryutoserver
└─ /srv/shared/remider
    │
    │ bin/atcoder-abc-reminder
    │ state.db
    └─ release-info.txt

```

ソースから生成した実行ファイルは一時ファイルへ転送し、ハッシュ値と実行権限を確認してから配置する。systemd の再起動またはサービス再読込は、転送・検証が成功した後に実行する。

3. 通知本文仕様

3.1 Discord へ送る本文

Discord API へ送る content は次の形式に固定する。{ROLE ID}は競プロロールの ID、{N}は ABC 番号、{START}と{END}は JST の HH:mm で置換する。<@&{ROLE_ID}>は Discord のロールメンションで、表示上は@競プロになる。先頭からロールメンション、空行、本文の順に配置する。本文内の # は Discord Markdown の見出し、[...]はコンテスト URL へのインラインリンクである。

<@&{ROLE_ID}>

AtCoder Beginner Contest {N}

本日 {START} ～ {END} に [AtCoder Beginner Contest {N}](https://atcoder.jp/contests/abc{N}) が開催されます。

皆さんぜひ参加しましょう! 🔥

例:

<@&123456789012345678>

AtCoder Beginner Contest 475

本日 21:00 ～ 22:40 に [AtCoder Beginner Contest 475](https://atcoder.jp/contests/abc475) が開催されます。

皆さんぜひ参加しましょう! 🔥

3.2 Discord リンクプレビュー

本文中の AtCoder インラインリンクにより、Discord の標準リンクプレビューを有効にする。画像に表示されている AtCoder のタイトル、説明、ロゴは AtCoder ページの OGP 情報を Discord が取得して表示するものであり、Bot 側でカスタム Embed を生成しない。

したがって、本文の Markdown とリンク先は固定できるが、プレビューの画像・説明文・表示タイミングは Discord または AtCoder 側の仕様に依存する。Discord API リクエストではリンクプレビューを抑制するフラグを指定しない。

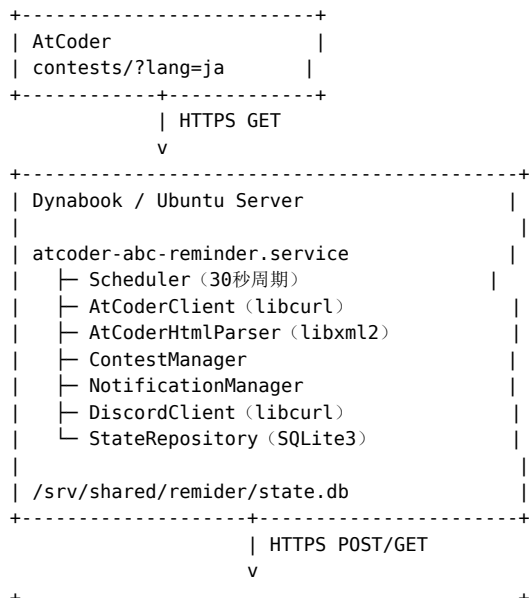
3.3 開始・終了時刻

画像の本文に終了時刻が必要なため、Contest モデルに endTime を追加する。AtCoder 一覧から開始時刻と開催時間を取得し、次の式で算出する。

$endTime = startTime + duration$

開催時間を確実に取得できない場合は、推測値や ABC 固有の固定値で補完せず、候補を無効として通知しない。開始・終了時刻の双方が検証できた場合だけ通知本文を生成する。

4. システム構成



```
| Discord REST API |
| POST /api/v10/channels/{id}/messages |
| GET /api/v10/channels/{id}/messages |
+-----+

```

Ubuntu Server 側でネットワーク接続状態を確認する機能は実装しない。HTTP 要求が失敗した場合のみ、その要求の失敗として処理し、保存状態を維持してバックオフ後に再試行する。

5. 実行環境・使用技術

5.1 対象環境

- Ubuntu Server LTS を搭載した Dynabook
- x86_64 Linux を第一対象とする
- C++17
- CMake
- systemd
- OpenSSL/TLS(libcurl 経由)

Ubuntu の具体的なバージョンは実装時に対象機体の LTS へ固定する。ホストのタイムゾーン設定に依存しないため、アプリケーション内部では常に JST を明示して扱う。

5.2 使用ライブラリ

ライブラリ	用途	採用理由
libcurl	HTTPS GET/POST、タイムアウト、HTTP ヘッダー取得	Linux で標準的で、TLS を任せられる
libxml2	不完全 HTML を含む AtCoder ページ解析	正規表現のみより誤解析しにくい
SQLite3	通知状態・現在コンテストの永続化	単一ファイル、トランザクション、無料
nlohmann/json	Discord JSON 本文の生成・レスポンス解析	JSON エスケープと解析を手作業にしない
OpenSSL	TLS 基盤	libcurl の HTTPS 検証に使用

依存関係は OS パッケージとして導入し、独自の常駐サーバーや有料 SDK は追加しない。標準ライブラリだけで安全に処理できる箇所は標準ライブラリを使用する。

5.3 開発・配置ディレクトリ

```
atcoder-abc-reminder/
├── CMakeLists.txt
├── README.md
├── .gitignore
├── include/
│   └── atcoder_reminder/
│       ├── atcoder_client.hpp
│       ├── atcoder_html_parser.hpp
│       ├── config.hpp
│       ├── discord_client.hpp
│       ├── domain.hpp
│       ├── http_client.hpp
│       ├── message_template.hpp
│       ├── ports.hpp
│       ├── reminder_policy.hpp
│       ├── state_repository.hpp
│       └── time_service.hpp
├── src/
│   ├── atcoder_client.cpp
│   ├── atcoder_html_parser.cpp
│   ├── discord_client.cpp
│   ├── http_client.cpp
│   ├── main.cpp
│   ├── message_template.cpp
│   ├── reminder_policy.cpp
│   ├── state_repository.cpp
│   └── time_service.cpp
├── tests/
│   └── test_main.cpp
└── deploy/
    ├── atcoder-abc-reminder.env.example
    └── atcoder-abc-reminder.service

```

5.4 デプロイ成果物

サーバーへ配置するものは次に限定する。

成果物	配置先	備考
実行ファイル	/srv/shared/remider/bin/atcoder-abc-reminder	CMake の Release ビルド

成果物	配置先	備考
SQLite	/srv/shared/remider/state.db	初回起動時に生成。既存 DB を上書きしない
リリース情報	/srv/shared/remider/release-info.txt	バージョン、ビルド日時、Git SHA 等
systemd 定義	/etc/systemd/system/atcoder-abc-reminder.service	SSH 先で sudo により更新
秘密情報	/etc/atcoder-abc-reminder/atcoder-abc-reminder.env	ソース・共有フォルダーへコピーしない

サーバー上の/srv/shared/remider は実行環境として扱い、ソースコードの編集場所にはしない。デプロイは次の順序で行う。

1. 開発環境でテスト、静的解析、Release ビルドを実行する。
2. 実行ファイルの SHA-256 を計算する。
3. SSH 経由で/srv/shared/remider/.staging/へ転送する。
4. サーバー上でハッシュ値、所有者、実行権限を確認する。
5. サービスを停止する。
6. 既存実行ファイルをバックアップしてから新しい実行ファイルへ切り替える。
7. systemctl daemon-reload 後にサービスを起動する。
8. systemctl is-active と journal で起動結果を確認する。

SQLite の state.db は通常のデプロイで上書きしない。スキーマ変更が必要な場合は、バックアップ取得と互換性確認を行ったうえで、アプリケーションのマイグレーション処理を使用する。

6. モジュール設計

6.1 domain

外部ライブラリに依存しない業務ルールを担当する。

```
namespace reminder {

using UnixSeconds = std::int64_t;

struct Contest {
    std::string id;          // abc475
    int number{};
    UnixSeconds startTime{};
    UnixSeconds endTime{};
    std::string url;        // 検証済みの正規URL
};

enum class NotificationKind {
    Normal,
    ScheduleChanged,
};

enum class NotificationStatus {
    None,
    Pending,
    Sent,
};

struct CurrentContest {
    Contest contest;
    UnixSeconds reminderTime{};
    NotificationStatus normalStatus{NotificationStatus::Pending};
    std::optional<UnixSeconds> normalSentForStartTime;
    NotificationStatus changeStatus{NotificationStatus::None};
    std::optional<UnixSeconds> changeSentForStartTime;
    std::optional<NotificationKind> pendingKind;
    std::string pendingMessageKey;
};

} // namespace reminder
```

ID は abc に数字が 1 つ以上続く形だけを許可する。URL は ID から生成し、AtCoder 以外のホストへ向けない。

6.2 time service

- std::chrono::system_clock から現在の Unix 秒を取得する。
- chrony または systemd-timesyncd 等、Ubuntu 側の時刻同期を前提とする。
- アプリケーション自身は NTP サーバーへ直接接続しない。
- 現在時刻の妥当性を起動時に確認する。
- 表示時だけ JST へ変換する。

Ubuntu のシステム時刻が未同期と判断される場合は、通知と AtCoder 候補確定を停止する。ただし Wi-Fi 接続状態を取得・監視する処理は持たない。

6.3 atcoder_client

<https://atcoder.jp/contests/?lang=ja> へ libcurl で GET する。

- ・ 接続タイムアウト: 30 秒
- ・ 転送タイムアウト: 30 秒
- ・ リダイレクト: 最大 3 回
- ・ 最終ホスト: atcoder.jp だけを許可
- ・ HTTP 2xx 以外: 取得失敗
- ・ レスポンス上限: 2MiB
- ・ TLS 証明書検証: 有効

ESP32 向けの 256KiB 制限は廃止するが、無制限読込は行わない。上限超過・取得失敗・解析失敗時には、現在の SQLite 状態を更新しない。

6.4 atcoder_html_parser

libxml2 の HTML パーサーを使用し、AtCoder ページを HTML 文書として解析する。外部ネットワーク参照は禁止し、取得済み本文だけを入力にする。

処理手順:

1. HTML を libxml2 で解析する。
2. コンテスト一覧の行またはリンク要素から /contests/abc[0-9]+ を探す。
3. 同じコンテスト行から開始日時と開催時間を取得する。
4. Contest を生成し、ID、番号、URL、開始・終了時刻を検証する。
5. 同じ ID が複数ある場合は、全フィールドが一致するものだけ重複排除する。一致しない場合はその ID を無効にする。
6. 現在時刻より未来で、開始時刻より終了時刻が後の候補だけを残す。
7. 開始時刻昇順、ID 昇順でソートする。

リンクの名称は ABC 判定に使用しない。スポンサー名付きの名称でも URL が ABC 形式なら採用する。

日時は、予定コンテスト表 (#contest-table-upcoming) の各行にある <time class="fixtime fixtime-full"> 要素のテキストから取得する。現在確認できる形式は YYYY-MM-DD HH:MM:SS+0900 である。開始日時、同じ行の開催時間(例:01:40) コンテストリンクを対応付け、endTime = startTime + duration で終了時刻を算出する。日時・開催時間のどちらかが抽出できない場合は候補を破棄する。HTML 構造が変わり、候補の妥当性を確定できない場合は解析エラーとする。

6.5 contest_manager

取得済み候補から次回 ABC を選択し、現在状態との差分を適用する。

候補なし (一覧構造を確認できない) -> 取得失敗扱いで状態維持・通知保留
新しいABC -> normal=pendingで新規保存
同一ABC・日時変更なし -> 状態を維持
同一ABC・通知前に日時変更 -> 開始/終了/通知時刻を更新
同一ABC・通知後に日時変更 -> 訂正通知をpendingにする
保存済みABCが未来なのに一覧から消失 -> 状態を維持し、その回の通知を保留
現在ABCの開始時刻を経過 -> 次回候補を取得できた場合のみ置換

予定表を取得できても完全な未来 ABC 候補が 0 件の場合は、HTML 構造変更や必要項目欠落の可能性を考慮して取得失敗扱いとする。現在状態を削除・置換せず、バックオフ後に再取得する。これにより、解析失敗時に古い予定を誤通知しない。

6.6 reminder_policy

```
constexpr std::int64_t kReminderBeforeSeconds = 2 * 60 * 60;
```

```
UnixSeconds reminderTime(const Contest& contest) {  
    return contest.startTime - kReminderBeforeSeconds;  
}
```

```
bool shouldSendNormal(const CurrentContest& state, UnixSeconds now) {  
    return state.normalStatus == NotificationStatus::Pending  
        && now >= state.reminderTime  
        && now < state.contest.startTime;  
}
```

開始時刻を過ぎてから初めて検出した通常通知は送らない。開始前に通知時刻を過ぎていれば、次回ループで即時送信する。

6.7 message_template

JST 日時を HH:mm で整形し、3 章の固定本文を生成する。コンテスト番号は整数から生成し、AtCoder HTML の名称を本文へ直接コピーしない。

Discord の JSON 本文は nlohmann/json で生成し、手作業の文字列連結による JSON 破壊を防ぐ。allowed_mentions.parse を空配列にしたうえで、設定された競プロロール ID だけを allowed_mentions.roles へ指定する。

```
{
  "allowed_mentions": {
    "parse": [],
    "roles": ["{ROLE_ID}"]
  }
}
```

6.8 discord client

使用エンドポイント:

POST https://discord.com/api/v10/channels/{channel_id}/messages
 GET https://discord.com/api/v10/channels/{channel_id}/messages?limit=100

POST 本文:

```
{
  "content": "...画像仕様の本文...",
  "allowed_mentions": { "parse": [], "roles": ["{ROLE_ID}"] },
  "nonce": "normal:abc475:...",
  "enforce_nonce": true
}
```

URL の埋め込みプレビューを表示させるため、Embed 抑制フラグは送らない。

Bot に必要な権限は次のとおりとする。

- View Channel
- Send Messages
- Read Message History(送信結果不明時の照合用)

競プロロールはメンション可能に設定する。メンション可能でないロールを Bot がメンションする場合は、Bot に Discord の Mention @everyone, @here, and All Roles 権限を付与する。

7. 永続化設計

7.1 SQLite

保存先:

/srv/shared/remider/state.db

SQLite を WAL モードで使い、状態更新をトランザクションで行う。DB ファイル、WAL ファイル、SHM ファイルを同一ディレクトリへ置く。

7.2 スキーマ

```
CREATE TABLE IF NOT EXISTS current_contest (
  singleton          INTEGER PRIMARY KEY CHECK (singleton = 1),
  contest_id         TEXT NOT NULL,
  contest_number     INTEGER NOT NULL,
  start_time         INTEGER NOT NULL,
  end_time           INTEGER NOT NULL,
  reminder_time      INTEGER NOT NULL,
  normal_status      TEXT NOT NULL CHECK (normal_status IN ('pending', 'sent')),
  normal_sent_at_start INTEGER,
  change_status      TEXT NOT NULL CHECK (change_status IN ('none', 'pending', 'sent')),
  change_sent_at_start INTEGER,
  pending_kind       TEXT CHECK (pending_kind IN ('normal', 'change')),
  pending_key        TEXT,
  last_fetched_at    INTEGER,
  updated_at         INTEGER NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS notification_history (
  message_key        TEXT PRIMARY KEY,
  contest_id         TEXT NOT NULL,
  kind               TEXT NOT NULL CHECK (kind IN ('normal', 'change')),
  start_time         INTEGER NOT NULL,
  status             TEXT NOT NULL CHECK (status IN ('pending', 'sent', 'skipped')),
  discord_message_id TEXT,
  updated_at         INTEGER NOT NULL
);
```

notification_history はコンテストが次回へ移行した後も通知キーを保持し、再起動やタイムアウト後の照合に使用する。

7.3 通知状態の更新

通常通知の送信前に、同一トランザクションで notification_history.status = 'pending' と current_contest.pending_kind = 'normal' を保存する。Discord 送信成功後に履歴と現在状態を sent へ更新する。

日時変更通知も同様に kind = 'change' の別キーで管理する。同一コンテストが複数回延期された場合は、変更後の開始時刻ごとに別キーを生成する。

保存成功前に送信済みと確定しない。SQLite 更新に失敗した場合は、次回ループで送信前の状態を再確認する。

7.4 外部送信との整合性

Discord 送信と SQLite 更新は同一トランザクションではないため、理論上の完全な Exactly-once は実現できない。本設計では次を組み合わせる。

- 1. 送信前に保留状態を保存
- 2. Discord nonce と enforce_nonce を利用
- 3. タイムアウト時に直近メッセージを取得し通知キーを照合
- 4. 再起動時も保留キーを照合
- 5. 照合できるまで通知済みへ変更しない

照合で既存メッセージが見つかった場合は、Discord メッセージ ID を履歴へ保存して送信済みとする。

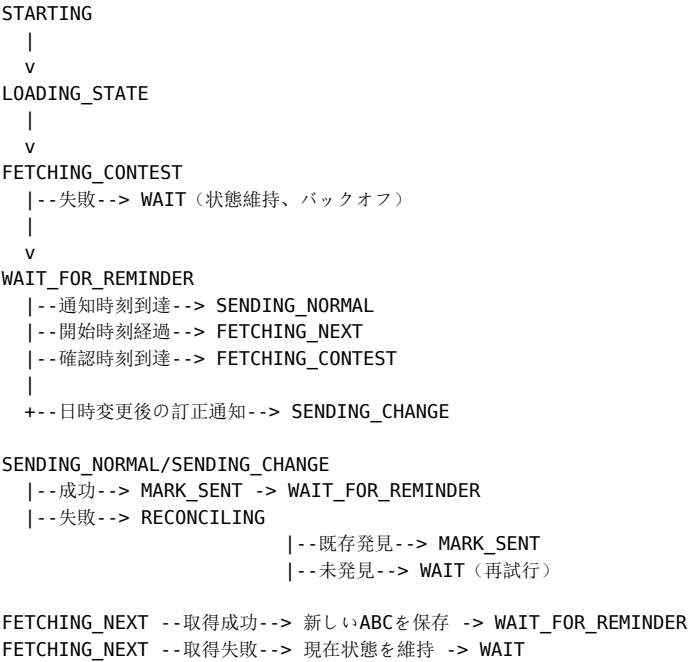
8. スケジュールと状態遷移

8.1 AtCoder 確認間隔

次回 ABC までの残り時間	確認間隔
24 時間超	60 分
6 時間超、24 時間以下	15 分
6 時間以下、開始前	5 分
対象なし	60 分

起動直後、前回取得失敗後のバックオフ完了時、時刻同期完了時は即時取得する。通知時刻の判定は 30 秒周期の内部時計だけで実施し、AtCoder ヘアクセスしない。

8.2 状態



8.3 起動時

- 1. systemd からプロセスを起動する。
- 2. SQLite を開き、スキーマを検証する。
- 3. 保留通知があれば、最初に Discord 履歴を照合する。
- 4. OS 時刻が有効であることを確認する。
- 5. AtCoder 一覧を即時取得する。
- 6. 現在状態と候補を比較し、日時変更または次回移行を適用する。
- 7. 30 秒周期のループを開始する。

9. エラー処理

9.1 ネットワーク・HTTP

Wi-Fi 接続確認は行わないが、HTTP 要求の失敗は処理する。

エラー	処理
DNS、接続、TLS、タイムアウト	5 分後から指数バックオフで再試行、最大 1 時間
AtCoder HTTP 5xx	保存状態を維持して再試行
AtCoder HTTP 4xx	URL・アクセス仕様をログし、30 分以上の間隔で再試行

エラー	処理
Discord 429	Retry-After 以上待機して再試行
Discord 5xx	指数バックオフで再試行
Discord 400	本文・リクエスト不正。未送信状態を保持し、30 分間隔でログ
Discord 401/403	Token・チャンネル・権限の問題。未送信状態を保持し、30 分間隔でログ
HTML 解析エラー	現在状態を更新せず、次回取得
SQLite エラー	通知済みにせず、エラー終了または次回処理へ回す

9.2 プロセス障害

- ・最上位で例外を捕捉し、秘密情報を含めずに journal へ出力する。
- ・予期しない終了時は systemd の Restart=on-failure で再起動する。
- ・同時実行を防ぐため、systemd のサービスを 1 インスタンスに限定する。
- ・SQLite のトランザクション途中で終了しても、SQLite のロールバックに任せる。

10. セキュリティ設計

10.1 秘密情報

Bot Token、Wi-Fi Password、Channel ID はソースコードや Git へ記録しない。Ubuntu 上の root 所有ファイルへ保存する。

```
/etc/atcoder-abc-reminder/atcoder-abc-reminder.env
```

権限:

```
root:root 600
```

サービス専用ユーザーだけが読み取れるよう、実運用時には LoadCredential または サービス専用の EnvironmentFile を使用する。Token をコマンドライン引数に置かない。

10.2 systemd サンドボックス

サービス定義では次を設定する。

```
User=abc-reminder
Group=abc-reminder
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/srv/shared/reminder
Restart=on-failure
RestartSec=30
```

ネットワーク通信に必要な権限以外を付与しない。ログへ Token、Password、Authorization ヘッダー、HTML 本文を出力しない。

10.3 入力検証

- ・AtCoder URL は <https://atcoder.jp/contests/abc>[0-9]+ に限定する。
- ・ABC 番号は整数として解析し、異常に大きい値を拒否する。
- ・日時・開催時間が不正な候補は採用しない。
- ・Discord 本文は固定テンプレートと検証済み値だけで生成する。
- ・allowed_mentions.parse を空配列にし、競プロロール ID だけを allowed_mentions.roles へ指定する。
- ・外部 HTML から取得した文字列を SQL へ連結せず、プレースホルダーを使う。

11. systemd 配置設計

サービスファイル例:

```
[Unit]
Description=AtCoder ABC Discord Reminder Bot
After=network-online.target

[Service]
Type=simple
ExecStart=/srv/shared/reminder/bin/atcoder-abc-reminder
EnvironmentFile=/etc/atcoder-abc-reminder/atcoder-abc-reminder.env
User=abc-reminder
Group=abc-reminder
WorkingDirectory=/srv/shared/reminder
Restart=on-failure
RestartSec=30
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/srv/shared/reminder
```


[Install]
WantedBy=multi-user.target

network-online.target は起動順序の宣言にのみ使用する。アプリケーションは Wi-Fi 接続確認を行わず、HTTP の成否で通信可能性を判断する。

Dynabook を通知装置として常時稼働させるため、Ubuntu の自動サスペンド・休止を無効にする設定を導入手順へ含める。これはアプリケーションの Wi-Fi 判定とは別の、機体の電源管理設定である。

12. テスト設計

12.1 単体テスト

GoogleTest 等の無料テストフレームワーク、または CMake で導入できる最小テストランナーを使用する。

対象	確認内容
URL 判定	abc475 を採用し、arc475、名称だけの ABC、別ホストを除外
HTML 解析	正常、スポンサー名、重複、日時欠落、構造変更、巨大本文
日時	JST、ISO 形式、無効日付、日跨ぎ、終了時刻計算
次回選択	過去・現在・未来・複数候補・同時刻
通知時刻	開始 7200 秒前、24 時間境界、6 時間境界
本文	画像仕様の改行、番号、開始・終了時刻、URL、絵文字
変更前	通常通知未送信時に通知時刻だけ再計算
変更後	訂正通知を 1 回、再変更時は新日時ごとに 1 回
SQLite	トランザクション、再起動復元、保留・送信済み状態
Discord	2xx、429、5xx、タイムアウト、履歴照合、JSON エスケープ

12.2 統合テスト

libcurl、Discord、AtCoder を Fake サーバーへ差し替え、次を実行する。

- ・ 起動から次回 ABC 取得・保存
- ・ 通知時刻到達から Discord POST・SQLite 更新
- ・ POST タイムアウト後に履歴から既存メッセージを発見
- ・ AtCoder 取得失敗後も既存の通知時刻を維持
- ・ 通知前の日時変更
- ・ 通知後の訂正通知
- ・ プロセス再起動後の保留通知再照合
- ・ コンテスト終了後の次回 ABC 移行
- ・ 連続 HTTP 失敗からのバックオフ復旧

12.3 実機・運用テスト

Dynabook 上で次を確認する。

- ・ systemctl enable --now で自動起動する
- ・ 24 時間以上連続稼働する
- ・ systemd による異常終了後の自動再起動
- ・ Ubuntu 再起動後の SQLite 状態復元
- ・ 自動サスペンド・休止無効化後に通知時刻まで稼働する
- ・ AtCoder 実ページから開始・終了時刻を取得する
- ・ Discord の本文とリンクプレビューが画像仕様になる
- ・ 通知後に開催日時を変更した場合、訂正通知が送られる

13. 要件トレーサビリティ

要件	設計対応
FR-01 Wi-Fi 接続	変更によりアプリケーションでの接続確認を対象外とする。OS のネットワークを利用する。
FR-02 時刻取得	Ubuntu の時刻同期と C++時刻サービス、JST 変換
FR-03 AtCoder 取得	libcurl による HTTPS GET と libxml2 解析
FR-04 次回 ABC	ContestManager の未来候補ソート
FR-05 2 時間前	startTime - 7200
FR-06 Discord 通知	Discord REST API POST
FR-07 通知内容	画像仕様の固定本文、AtCoder URL、標準リンクプレビュー
FR-08 日時変更	同一 ID の開始・終了時刻比較
FR-09 通知後変更	notification_history と訂正通知
FR-10 重複防止	SQLite、nonce、履歴照合

要件	設計対応
FR-11 状態保存	SQLite のローカルファイルとトランザクション
FR-12 次回移行	開始後に取得成功した次回候補へ置換
NFR-01 軽量性	ESP32 制約を解除。ただし本文・候補・HTTP 応答に上限を設ける
NFR-02 常時稼働	systemd、自動再起動、タイムアウト
NFR-03 再起動耐性	SQLite、保留通知、履歴照合
NFR-04 障害耐性	HTTP/Discord/HTML/SQLite 別の失敗処理
NFR-05 負荷抑制	60 分/15 分/5 分の段階的確認間隔

FR-01 はユーザー指定により「Wi-Fi 接続状態を確認しない」へ変更した。Wi-Fi が利用できるかどうかを検出する処理は実装しないが、HTTP 要求自体の失敗は通知保護のため処理する。

14. 費用・運用

14.1 継続費

- ・ C++、CMake、libcurl、libxml2、SQLite3、nlohmann/json、systemd: 無料で利用できるソフトウェアを使用する。
- ・ AtCoder: 公開ページを取得するだけで、有料 API を利用しない。
- ・ Discord: 既存の Bot、サーバー、チャンネルを利用し、Gateway 常駐基盤や有料ホスティングを使わない。
- ・ クラウドサーバー、DB、監視サービス: 使用しない。

月額固定費は増加させない。Dynabook の購入費や必要な周辺機器は一括購入費であり、電気代は機体を常時稼働させることによる変動費として発生する。

14.2 バックアップ

SQLite ファイルは通知履歴を含むため、サービス停止後に state.db、state.db-wal、state.db-shm を整合した状態でバックアップする。バックアップ先は同一ディスク上だけにせず、必要に応じてユーザーが手動で外部媒体へコピーする。

15. 作業単位

UOW-01 C++ビルド骨格

- ・ CMake、C++17、libcurl、libxml2、SQLite3、JSON ライブラリを導入する。
- ・ main.cpp から起動・終了ログを出す。
- ・ 完了条件: Ubuntu Server 上でビルドと起動ができる。

UOW-02 ドメイン・時刻・本文

- ・ Contest、通知時刻、終了時刻、画像仕様の本文を実装する。
- ・ 完了条件: 日時境界と本文比較テストに通る。

UOW-03 AtCoder 取得・解析

- ・ libcurl と libxml2 で開始・終了時刻付き ABC 候補を取得する。
- ・ 完了条件: フィクスチャと実ページで妥当な候補だけを抽出する。

UOW-04 SQLite 永続化

- ・ スキーマ、トランザクション、通知履歴、再起動復元を実装する。
- ・ 完了条件: 送信前・送信後・保存失敗のテストに通る。

UOW-05 Discord 通知

- ・ REST POST、履歴照合、429、再試行、リンクプレビューを実装する。
- ・ 完了条件: Discord の実チャンネルで本文とプレビューを確認する。

UOW-06 systemd・長期稼働

- ・ サービスユーザー、環境ファイル、systemd、自動再起動、電源管理を設定する。
- ・ 完了条件: Ubuntu 再起動・プロセス異常終了・24 時間稼働テストに通る。

16. 既知の制約・残存リスク

1. AtCoder 一覧 HTML の構造変更時は候補を安全側で無効にする。実装更新が必要になる。
2. Discord 送信と SQLite 保存は別システムのため、数学的な Exactly-once は保証できない。nonce と履歴照合で重複を最小化する。
3. Discord のリンクプレビューは外部サービスの生成結果であり、画像・説明・表示の完全一致は保証できない。本文と URL 配置は画像仕様に固定する。
4. Dynabook が電源断、休止、故障した時間帯は通知できない。常時給電と自動サスペンド無効化を運用条件とする。
5. Bot Token が漏洩した場合は Discord Developer Portal で再生成し、環境ファイルを更新する。
6. Ubuntu の時刻同期が壊れている場合は誤通知防止のため通知しない。

17. 完成条件

- ・ Ubuntu Server 搭載 Dynabook 上で C++17 プログラムが常駐する。
- ・ AtCoder から ABC 番号、開始時刻、終了時刻を自動取得する。
- ・ ABC 番号の手動入力が必要である。

- 開始 2 時間前に画像仕様の本文を Discord へ送信する。
- 本文中の AtCoder インラインリンクにより標準リンクプレビューを表示できる。
- 開催日時変更を検知し、通知前は予定時刻を更新、通知後は訂正通知を行う。
- 同じ通知キーの通知を重複させない。
- Ubuntu・プロセス再起動後も SQLite から状態を復元する。
- コンテスト終了後、次回 ABC へ自動移行する。
- HTTP 障害、HTML 解析失敗、Discord 失敗で誤った状態へ遷移しない。
- Discord Gateway、クラウドサーバー、月額サービスを必要としない。